ภาพรวมของบริการ (Service Overview)

https://github.com/Nattanan-Doangjinda/CS366_ShelterService

Shelter Service

1. Service Owner

นายณัฐนนท์ ดวงจินดา รหัสนักศึกษา 6609611923 ภาคปกติ

2.Service Purpose

Shelter Service เป็นบริการที่รับผิดชอบการจัดการข้อมูลศูนย์พักพิง ความจุที่เหลืออยู่ พิกัดที่ตั้ง และสภาพเส้นทางการเข้าถึง (Access Level) แบบ Real-time โดยทำหน้าที่เป็นแหล่งข้อมูลหลักในการค้นหา อัปเดตสถานะ และจัดสรรผู้ประสบภัยไปยังศูนย์พักพิงที่เหมาะสมที่สุดและสามารถเดินทางไปได้จริง เพื่อสนับสนุนการทำงานของเจ้าหน้าที่ในระบบตอบสนองภัยพิบัติ

3.Pain Point ที่แก้ไข

ในภาวะฉุกเฉิน (เช่น น้ำท่วม) ข้อมูลศูนย์พักพิงมักไม่อัปเดต เจ้าหน้าที่ไม่รู้ว่าศูนย์พักพิงไหน 'เต็ม' หรือ 'ยังว่าง' รวมถึงไม่รู้ว่าที่ไหนอยู่ใกล้และเส้นทางปลอดภัยพอที่จะเข้าไปได้หรือไม่ ทำให้เกิดการส่งคนไปผิดที่ เกิดความแออัด วุ่นวาย และผู้ประสบภัยอาจติดค้างระหว่างทาง บริการนี้จึงออกแบบมาเพื่อให้การอัปเดตความจุและสภาพเส้นทางมีความถูกต้องทันที (Real-time decision support) หากไม่มี Service นี้ ระบบโดยรวม (Ecosystem) จะขาดแหล่งอ้างอิงข้อมูลความจุที่เป็นศูนย์กลาง (Single Source of Truth) ส่งผลให้ระบบประสานงานอพยพทำงานผิดพลาด สั่งการทับซ้อนกันจนเกิด Race Condition และส่งผู้ประสบภัยเข้าไปในพื้นที่อันตรายได้

4. Target Users

- เจ้าหน้าที่ศูนย์พักพิง (Shelter Staff)
- ทีมกู้ภัย / ผู้ประสานงาน (Rescue Team / Dispatcher)
- ระบบประสานงานอพยพส่วนกลาง (Evacuation Coordination Service - ใช้งานผ่าน API)

5. Service Boundary

- In-scope Responsibilities (สิ่งที่บริการนี้รับผิดชอบ)

    - จัดเก็บข้อมูลศูนย์พักพิงและพิกัดที่ตั้ง (Shelter Master Data & Location)
    - จัดการสถานะ ความจุที่เหลืออยู่ และระดับความปลอดภัยของเส้นทาง (Operational State & Access Level)
    - คำนวณระยะทางเบื้องต้นและคัดกรองศูนย์พักพิงที่เหมาะสม (Distance & Availability Filtering)
    - บังคับใช้กติกาการจัดสรรความจุ ป้องกันการรับคนเกินกำหนด (Capacity Allocation Rules)

- Out-of-scope / Not Responsible For (ไม่รับผิดชอบ)
    - การจัดการข้อมูลเหตุการณ์ภัยพิบัติหลัก (Incident Master Data)
    - การวางแผนระบบนำทางแบบละเอียด (Turn-by-turn Navigation)
    - ข้อมูลส่วนบุคคลเชิงลึกของผู้เข้าพัก (Personal Identifiable Information ของผู้ประสบภัย)

6. Autonomy / Decision Logic

บริการมีความเป็นอิสระในการตัดสินใจเกี่ยวกับ:

- การยืนยันหรือปฏิเสธการ Check-in จะอิงจากความจุที่เหลือ (Capacity Limit) อย่างเด็ดขาดเพื่อรักษามาตรฐานความปลอดภัยและป้องกันความแออัดขั้นวิกฤตในศูนย์พักพิง โดยไม่ต้องเสียเวลารอเจ้าหน้าที่มนุษย์อนุมัติ)
- การปรับเปลี่ยนสถานะของศูนย์พักพิงโดยอัตโนมัติ (เช่น เปลี่ยนสถานะเป็น FULL ทันทีเมื่อ current_occupancy เท่ากับ total_capacity)

การตัดสินใจอิงจาก:

- ระยะทางจากพิกัดผู้ประสบภัย (Distance)
- จำนวนความจุที่ยังว่าง (Available Capacity)
- ความปลอดภัยของเส้นทางเข้าถึง (Access Level เช่น SAFE > DIFFICULT)

บริการสามารถตัดสินใจได้เองภายใต้ business rules ที่กำหนด <u>โดยไม่ต้องรอ/ต้องรอ</u>การอนุมัติจากมนุษย์ในกรณีปกติ

7. Owned Data

- Shelter Master Data: ข้อมูลพื้นฐานและพิกัด (Lat/Long) ของศูนย์พักพิง บริการนี้ต้องเป็นเจ้าของเพราะเป็นแกนหลักของการค้นหา
- Operational Capacity State: ข้อมูลความจุ จำนวนผู้เข้าพักปัจจุบัน (Occupancy) และสถานะ (Status) ซึ่งเปลี่ยนแปลงตลอดเวลาและต้องใช้ตัดสินใจรับคน
- Access Condition: ข้อมูลสภาพเส้นทางเข้าสู่ศูนย์ (access_level, access_note) เพื่อป้องกันการส่งคนเข้าไปในพื้นที่อันตราย

8. Linked Data (Reference Only)

- User/Staff ID: เพื่อตรวจสอบสิทธิ์ว่าใครเป็นผู้อัปเดตสถานะหรือสภาพเส้นทาง
- Incident ID (ถ้ามีระบบส่วนกลาง): อ้างอิงว่าการเปิดศูนย์พักพิงนี้ผูกกับเหตุการณ์ภัยพิบัติใด (เช่น รหัสเหตุการณ์น้ำท่วมเชียงราย) โดยบริการนี้จะไม่เก็บรายละเอียดของเหตุการณ์ไว้เอง

9. Non-Functional Requirements

- การทำรายการที่ส่งผลต่อความจุ (Occupancy Update) ต้องใช้กลไกป้องกัน Race Condition (เช่น Database Transaction หรือ Optimistic Locking) เพื่อป้องกันการส่งคนไปพร้อมกันจนเกินความจุ
- API สำหรับการค้นหาศูนย์ (Find Available Shelter) แบบ Synchronous ต้องตอบสนองอย่างรวดเร็ว (Low Latency) และมีการ Authentication
- ระบบต้องไม่ Crash เมื่อได้รับข้อมูลพิกัด (Lat/Long) ที่ผิดรูปแบบ หรือข้อมูลสถานะที่ไม่ถูกต้อง และตอบกลับด้วย Error Code ที่เหมาะสม (เช่น 409 Conflict เมื่อที่พักเต็ม)
- รองรับการ Retry ไม่เกิน 3 ครั้ง ในกรณีที่ระบบ Network มีปัญหาขณะอัปเดตข้อมูลความจุ

Synchronous Function Contract

Base URL:

API Contract #1: Find Available Shelter

ข้อมูลทั่วไป

- Name: Find available shelters by location
- Method: GET
- Path: /v1/shelters
- Type: Synchronous

คำอธิบาย

ใช้ค้นหาและกรองศูนย์พักพิงที่ยังมีสถานะเปิดรับ (OPEN) โดยคำนวณระยะทางจากพิกัดผู้ประสบภัย

เนื่องจากผู้เรียกต้องการผลลัพธ์แบบ Real-time เพื่อใช้ตัดสินใจช่วยเหลือหน้างานทันที โดยยอมรับข้อจำกัดเรื่อง Timeout (30s) ได้หากเครือข่ายมีปัญหา ดีกว่าการได้รับข้อมูลที่ไม่อัปเดต

Request

Path/Query Params

- user_lat (decimal, required): พิกัดละติจูดของผู้ประสบภัย
- user_long (decimal, required): พิกัดลองจิจูดของผู้ประสบภัย
- radius_km (number, optional, default = 10): รัศมีการค้นหา

Headers

- Accept: application/json

Body: ไม่มี

Response

Success: 200


```
{
  "items": [
    {
      "shelterId": "SHE-1234",
      "name": "โรงเรียนวัดเวฬุวัน",
```

"distanceKm": 1.2,
"accessLevel": "DIFFICULT",
"accessNote": "รถเล็กเข้าไม่ได้ น้ำท่วม 30cm",
"capacityAvailable": 55,
"status": "OPEN"
}
]
}

Error: 400

{
"error": {
"code": "VALIDATION_ERROR",
"message": "user_lat and user_long are required",
"traceId": "uuid-112233"
}
}

Dependency / Reliability

- ไม่เรียก service อื่น
- Idempotent
- Timeout: 30s

API Contract #2: Update Shelter Access & Status

ข้อมูลทั่วไป

- Name: Update shelter condition
- Method: PATCH
- Path: /v1/shelters/{shelterId}
- Type: Synchronous

คำอธิบาย

ใช้สำหรับเจ้าหน้าที่หน้างานในการปรับสถานะความจุ (กรณีฉุกเฉิน) และอัปเดตสภาพเส้นทางการเข้าถึง (Access Level) ระหว่างเกิดเหตุการณ์ เพื่อป้องกันการส่งคนไปในพื้นที่อันตราย

Request

Path/Query Param

- shelterId (string, required)

Headers

- Content-Type: application/json
- Authorization: Bearer <token>

Body

```
{
  "capacityAvailable": 40,
  "status": "OPEN",
  "accessLevel": "DANGEROUS",
  "accessNote": "สะพานหลักขาด ให้เข้าทางซอย 2"
}
```

Validation

- capacityAvailable >= 0
- status ต้องเป็น [OPEN, FULL, CLOSED, MAINTENANCE]
- accessLevel ต้องเป็น [SAFE, DIFFICULT, DANGEROUS]

Response

Success: 200

```
{
  "shelterId": "SHE-1234",
  "capacityAvailable": 40,
  "status": "OPEN",
  "accessLevel": "DANGEROUS",
  "updatedAt": "2026-02-16T11:20:00Z"
```

}

Error: 404

{
  "error": {
    "code": "NOT_FOUND",
    "message": "Shelter not found",
    "traceId": "uuid-445566"
  }
}

Dependency / Reliability

- ไม่เรียก service อื่น
- ถือเป็น Idempotent update

API Contract #3: Get Shelter Details

ข้อมูลทั่วไป

- **Name:** Get shelter details by ID
- **Method:** GET
- **Path:** /v1/shelters/{shelterId}
- **Type:** Synchronous

คำอธิบาย ใช้สำหรับดึงข้อมูลรายละเอียดทั้งหมดของศูนย์พักพิงแบบเจาะจง 1 แห่ง โดยใช้รหัส shelterId เพื่อนำไปแสดงผลบนหน้าจอแอปพลิเคชัน (เช่น หน้าดูรายละเอียดศูนย์พักพิงก่อนที่ผู้ประสบภัยหรือเจ้าหน้าที่จะกดยืนยันการจอง)

Request

- **Path/Query Params**
    - shelterId (string/UUID, required): รหัสอ้างอิงเฉพาะของศูนย์พักพิง
- **Headers**
    - Accept: application/json
- **Body:** ไม่มี

Response

- **Success: 200**


```
{
  "shelterId": "123e4567-e89b-12d3-a456-426614174000",
  "name": "โรงเรียนวัดเวฬุวัน",
  "latitude": 13.7563,
  "longitude": 100.5018,
  "capacityTotal": 100,
  "capacityAvailable": 55,
  "status": "OPEN",
  "accessLevel": "DIFFICULT",
  "accessNote": "รถเล็กเข้าไม่ได้ น้ำท่วม 30cm",
  "lastUpdatedAt": "2026-02-17T10:30:00Z"
}
```


- **Error: 404**


```
{
  "error": {
    "code": "NOT_FOUND",
    "message": "Shelter not found with the provided ID",
    "traceId": "uuid-778899"
  }
}
```


Dependency / Reliability

- ไม่เรียก service อื่น
- ถือเป็น Idempotent (เรียกซ้ำกี่ครั้งก็ได้ผลเหมือนเดิม)
- Timeout: 30s

API Contract #4: Register New Shelter

ข้อมูลทั่วไป

- **Name:** Register new shelter
- **Method:** POST
- **Path:** /v1/shelters
- **Type:** Synchronous

คำอธิบาย ใช้สำหรับเจ้าหน้าที่ส่วนกลางหรือแอดมินในการ "เพิ่มข้อมูลศูนย์พักพิงแห่งใหม่" เข้าสู่ระบบฐานข้อมูลหลัก (Shelter Master Data) เมื่อมีการตั้งศูนย์อพยพชั่วคราวเพิ่มเติมในพื้นที่ประสบภัย

Request

- **Path/Query Params:** ไม่มี
- **Headers**
    - Content-Type: application/json
    - Authorization: Bearer <token>
- **Body**

JSON

```
{
  "name": "ศูนย์เยาวชนเทศบาล",
  "latitude": 13.7485,
  "longitude": 100.4960,
  "capacityTotal": 200
}
```

Validation

- name ต้องไม่เป็นค่าว่าง (required)
- latitude และ longitude ต้องเป็นตัวเลขทศนิยมที่อยู่ในช่วงพิกัดที่ถูกต้อง (required)
- capacityTotal ต้องเป็นจำนวนเต็มที่มากกว่า 0 (required)

Response

- **Success: 201 Created**

JSON


```
{
  "message": "Shelter registered successfully",
  "data": {
    "shelterId": "323e4567-e89b-12d3-a456-426614174002",
    "name": "ศูนย์เยาวชนเทศบาล",
    "status": "OPEN",
    "capacityAvailable": 200,
    "createdAt": "2026-02-17T11:00:00Z"
  }
}
```


- **Error: 400**

JSON


```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "capacityTotal must be greater than 0",
    "traceId": "uuid-990011"
  }
}
```


Dependency / Reliability

- ไม่เรียก service อื่น
- ไม่เป็น Idempotent (กดส่งซ้ำอาจทำให้เกิดข้อมูลเบิ้ลได้)
- Timeout: 30s

Asynchronous Function Contract

Message Contract #1: RequestShelterReservation

ข้อมูลทั่วไป

- Message Name: ShelterReservationRequested
- Interaction Style: Request-Async Response (via Message Broker)
- Producer: Evacuation Service
- Consumer: Shelter Service
- Channel/Queue: shelter.reservation.commands.v1
- Version: v1

คำอธิบาย

ส่งคำขอจองความจุศูนย์พักพิงสำหรับผู้ประสบภัยแบบ Asynchronous โดยระบบจะนำคำขอเข้าคิวเพื่อประมวลผลตามลำดับ (ช่วยป้องกันปัญหา Race Condition) และจะส่งผลการจอง (ยืนยัน/ปฏิเสธ) กลับไปให้ผู้ส่งในภายหลังผ่านช่องทาง Event Notification

Reliability & Failure Handling: ออกแบบโดยคิดเผื่อกรณีระบบล้มเหลว หากการประมวลผลคำขอขัดข้อง ระบบจะมีกลไก Retry และหากล้มเหลวซ้ำจะนำข้อความส่งเข้า Dead Letter Queue (DLQ) เพื่อไม่ให้ระบบรวมหยุดชะงัก

Request

Message Headers (required)

- messageType: ShelterReservationRequested
- messageId: UUID (ใช้เป็น Idempotency Key ป้องกันการประมวลผลซ้ำ)
- replyTo: evac.reservation.results.v1 (Queue/Topic ที่ให้ส่งผลลัพธ์กลับ)
- sentAt: ISO-8601 datetime
- traceId: string/uuid (optional)

Message Body

```
{
  "incidentId": "8b9e4b0d-3b7e-4a0d-9d9f-3f5c6a2a3e10",
  "shelterId": "SHE-1234",
  "peopleCount": 15,
  "priority": "HIGH",
  "requestedBy": "RescueTeam-Alpha"
}
```

Field Definition:

Field	Type	Required	Description
incidentId	UUID	Y	รหัสอ้างอิงเหตุการณ์ภัยพิบัติ (เช่น รหัสน้ำท่วมเชียงราย)
shelterId	string	Y	รหัสศูนย์พักพิงเป้าหมาย
peopleCount	integer	Y	จำนวนผู้ประสบภัยที่ต้องการจองพื้นที่
priority	enum	Y	ระดับความเร่งด่วน (LOW, NORMAL, HIGH)
requestedBy	string	Y	ชื่อหน่วยงานหรือ Service ผู้ส่งคำขอ

Validation Rules

1. incidentId ต้องไม่ว่าง และต้องผ่านการตรวจสอบรูปแบบ (Format Check) ให้เป็น UUID ที่ตรงกับ Central Incident Schema เท่านั้น
2. peopleCount ต้องเป็นจำนวนเต็มบวก (มากกว่า 0)
3. priority ต้องเป็น LOW, NORMAL หรือ HIGH เท่านั้น
4. messageId (จาก Header) ต้องไม่ซ้ำกับคำขอที่เคยประมวลผลไปแล้ว (Idempotency check)

Response

Message Headers (required)

- messageType: ShelterReservationConfirmed (กรณีสำเร็จ) หรือ ShelterReservationRejected (กรณีไม่สำเร็จ)
- messageId: UUID ของข้อความ Response นี้
- correlationId: ตรงกับ messageId ของ Request ต้นทาง (เพื่อให้ฝั่งส่งรู้ว่าเป็นคำตอบของคำขอไหน)
- sentAt: ISO-8601 datetime
- traceId: string/uuid (optional)

Success Message Body


```
{
  "incidentId": "8b9e4b0d-3b7e-4a0d-9d9f-3f5c6a2a3e10",
  "shelterId": "SHE-1234",
  "reservationId": "RSV-998877",
  "peopleReserved": 15,
  "capacityAvailableAfter": 25,
  "status": "CONFIRMED"
}
```


Reject/Error Message Body


```
{
  "incidentId": "8b9e4b0d-3b7e-4a0d-9d9f-3f5c6a2a3e10",
  "shelterId": "SHE-1234",
  "status": "REJECTED",
  "reasonCode": "INSUFFICIENT_CAPACITY",
  "reasonMessage": "Requested capacity exceeds available capacity. Only 5 spots left."
}
```


Field Definition:

Field	Type	Required	Description

incidentId	UUID	Y	รหัสเหตุการณ์ที่เกี่ยวข้อง
shelterId	string	Y	รหัสศูนย์พักพิงที่ทำการจอง
reservationId	string	N	รหัสการจองที่สร้างโดย Shelter Service (มีเฉพาะตอนสำเร็จ)
peopleReserved	integer	N	จำนวนคนที่ได้รับการจัดสรรจริง (มีเฉพาะตอนสำเร็จ)
capacityAvailableAfter	integer	N	ความจุที่เหลืออยู่หลังจากการจองนี้ (มีเฉพาะตอนสำเร็จ)
status	enum	Y	สถานะผลลัพธ์การจอง (CONFIRMED / REJECTED)
reasonCode	string	N	รหัสสาเหตุที่ถูกปฏิเสธ (Machine-readable)
reasonMessage	string	N	ข้อความอธิบายสาเหตุที่ถูกปฏิเสธ (Human-readable)

Validation Rules

1. correlationId (ใน Header) ต้องตรงกับ messageId ของ Request
2. ถ้า status = CONFIRMED ต้องมีข้อมูล reservationId และ peopleReserved
3. ถ้า status = REJECTED ต้องมีข้อมูล reasonCode และ reasonMessage
4. reasonCode ต้องใช้โค้ดมาตรฐานที่ระบบตกลงกันไว้ เช่น INSUFFICIENT_CAPACITY, SHELTER_CLOSED

Service Data

1) Shelter Master Data (Owned by this service)

ตารางสำหรับเก็บข้อมูลพื้นฐานและที่ตั้งของศูนย์พักพิง

Field Name	Type	Required	Description	Example
shelterId	UUID	Y (PK)	รหัสอ้างอิงเฉพาะของศูนย์พักพิง	123e4567-e89b-12d3-a456-426614174000
name	string	Y	ชื่อศูนย์พักพิง	โรงเรียนวัดเวฬุวัน
latitude	decimal	Y	พิกัดละติจูดที่ตั้งศูนย์พักพิง	13.7563
longitude	decimal	Y	พิกัดลองจิจูดที่ตั้งศูนย์พักพิง	100.5018
capacityTotal	integer	Y	ความจุสูงสุดทั้งหมดที่รองรับได้	100

2) Shelter Operational State (Owned by this service)

ตารางสำหรับจัดการสถานะปัจจุบัน ความจุที่เหลืออยู่ และสภาพเส้นทาง ซึ่งมีการอัปเดตตลอดเวลา

Field Name	Type	Required	Description	Example
shelterId	UUID	Y (FK)	รหัสอ้างอิงศูนย์พักพิง (เชื่อมกับ Master)	123e4567-e89b...
currentOccupancy	integer	Y	จำนวนผู้พักพิงในปัจจุบัน (ข้อมูลฟิลด์นี้มีการใช้งานพร้อมกันสูง (Concurrency) จึงต้องมีการบังคับใช้ Row-level Locking ในระดับ	45

			Database ทุกครั้งที่มีการอัปเดตเพื่อรักษาความถูกต้อง)	
status	enum	Y	สถานะเปิด/ปิด (OPEN/FULL/CLOSED/MAINTENANCE)	OPEN
accessLevel	enum	Y	ระดับความปลอดภัยของเส้นทาง (SAFE/DIFFICULT/DANGEROUS)	DIFFICULT
accessNote	string	N	หมายเหตุอธิบายสภาพเส้นทางเพิ่มเติม	รถเล็กเข้าไม่ได้ น้ำท่วม 30cm
lastUpdatedBy	string	N	ชื่อหรือ ID เจ้าหน้าที่ผู้แก้ไขข้อมูลล่าสุด	staff-007
lastUpdatedAt	datetime	Y	วันเวลาที่อัปเดตสถานะล่าสุด	2026-02-17T10:30:00Z

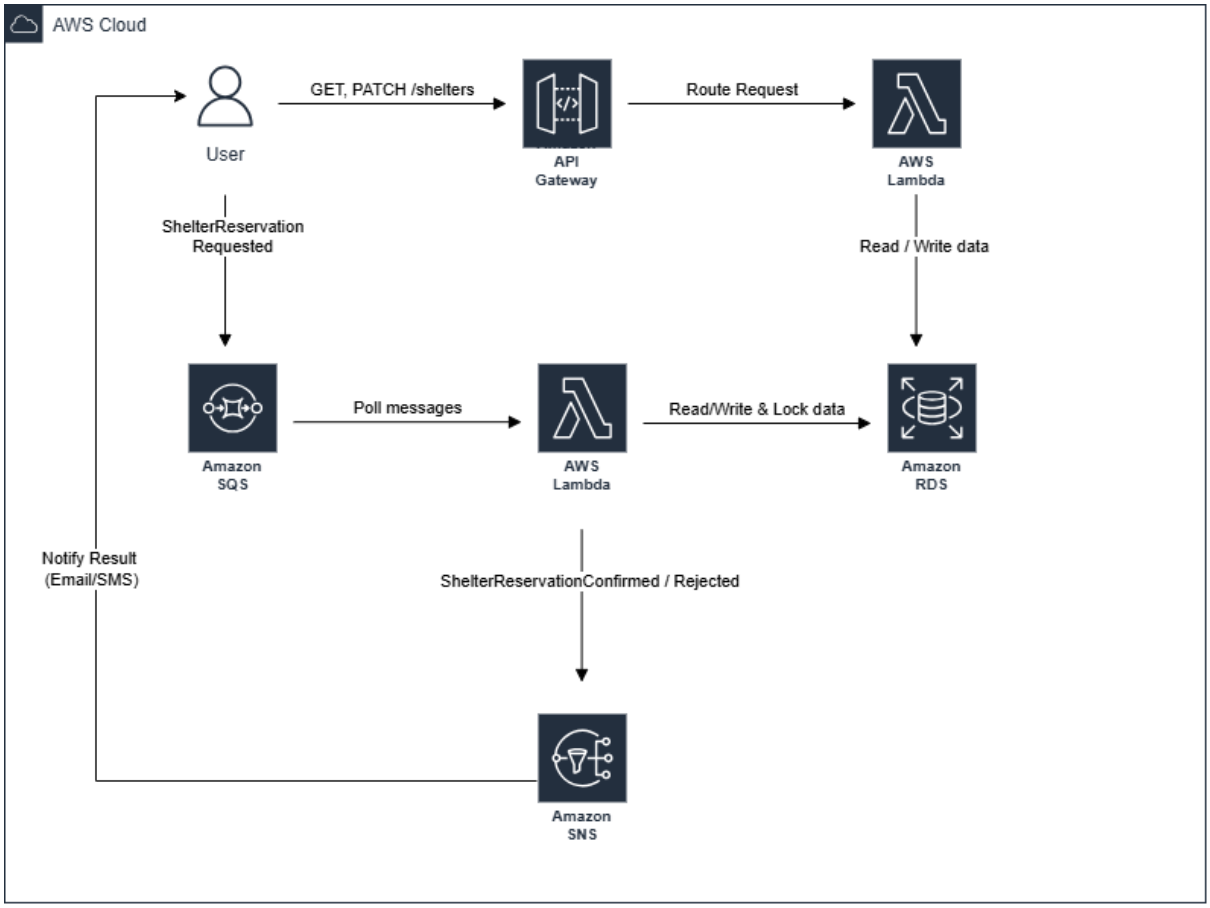
3) Reservation Data (Owned by this service)

ตารางสำหรับเก็บข้อมูลประวัติการจองที่เข้ามาทางระบบ Asynchronous Queue

Field Name	Type	Required	Description	Example
reservationId	string	Y (PK)	รหัสการจองที่ระบบสร้างขึ้น	RSV-00123
shelterId	UUID	Y (FK)	รหัสศูนย์พักพิงที่ถูกจอง	123e4567-e89b-12d3-a456-426614174000

incidentId	UUID	Y	อ้างอิงรหัสเหตุการณ์ภัยพิบัติ (Reference Only)	8b9e4b0d-3b7e-4a0d-9d9f-3f5c6a2a3e10
peopleReserved	integer	Y	จำนวนผู้ประสบภัยที่ยืนยันการจัดสรรพื้นที่	15
priority	enum	Y	ระดับความเร่งด่วน (LOW/NORMAL/HIGH)	HIGH
status	enum	Y	สถานะของคำขอจอง (CONFIRMED/REJECTED)	CONFIRMED
messageId	UUID	Y	รหัสอ้างอิงจากคิว เพื่อป้องกันการจองซ้ำ (Idempotency Key)	a1b2c3d4-e5f6-7g8h-9i0j-123456789abc
createdAt	datetime	Y	วันเวลาที่ทำการบันทึกการจอง	2026-02-17T10:32:00Z

Service Architecture



Components

- **Client / Upstream Services:** ผู้เรียกใช้งานระบบ เช่น แอปพลิเคชันของเจ้าหน้าที่ศูนย์พักพิง หรือ Evacuation Service
- **Amazon API Gateway:** จุดรับคำขอแบบ Synchronous ทำหน้าที่เป็นด่านหน้าในการรับ-ส่ง API (GET, PATCH) และตรวจสอบสิทธิ์
- **REST API Handler (AWS Lambda):** ประมวลผลตรรกะแบบเรียลไทม์ เช่น การค้นหาพิกัดและระยะทางของศูนย์พักพิง และการอัปเดตสถานะแบบด่วน
- **Amazon RDS (PostgreSQL):** ฐานข้อมูลเชิงสัมพันธ์ ทำหน้าที่เก็บ Master Data และสถานะความจุ **(เลือกใช้ Relational Database เพื่อประยุกต์ใช้ฟีเจอร์ Transaction และ Row-level Locking ในการป้องกันปัญหา Race Condition อย่างเด็ดขาด)**
- **Amazon SQS (shelter.reservation.commands.v1):** Message Queue สำหรับรับคำขอจองพื้นที่แบบ Asynchronous ทำหน้าที่เป็น Buffer เพื่อไม่ให้ระบบล่มเมื่อมีคำขอจำนวนมาก

- **Async Worker (AWS Lambda):** ดึงคำสั่งจาก SQS มาประมวลผลตามลำดับคิว เพื่ออัปเดตโควตาความจุใน Amazon RDS
- **Amazon SNS (evac.reservation.results.v1):** ช่องทางสำหรับกระจาย Event แจ้งเตือนผลลัพธ์การจอง (CONFIRMED / REJECTED) กลับไปยังระบบต้นทาง

Explanation

สถาปัตยกรรมของ Shelter Service ประยุกต์ใช้ AWS Serverless Ecosystem ผสมผสานกับ Relational Database เพื่อรองรับการทำงานทั้งแบบ Synchronous และ Asynchronous

สำหรับการทำงานปกติ เช่น การค้นหาศูนย์พักพิงที่เปิดรับและอยู่ใกล้ที่สุด (GET /shelters) ผู้ใช้งานจะส่งพิกัดตำแหน่ง (Latitude/Longitude) ของตนเองเข้ามาผ่าน Amazon API Gateway ระบบจะส่ง Request ต่อให้ AWS Lambda นำพิกัดดังกล่าวไปประมวลผล โดยจะเข้าไป Query ข้อมูลจาก Amazon RDS (PostgreSQL) ซึ่งมีความสามารถในการจัดการข้อมูลเชิงพื้นที่ (Spatial Data) ทำให้ระบบสามารถคำนวณระยะทาง กรองศูนย์พักพิงที่อยู่ในรัศมีที่กำหนด และเรียงลำดับสถานที่ที่ใกล้ที่สุดพร้อมข้อมูลสภาพเส้นทางได้อย่างแม่นยำและรวดเร็ว ก่อนส่งผลลัพธ์กลับไปให้ผู้ใช้งานแบบทันที

สำหรับการจองพื้นที่ศูนย์พักพิง ซึ่งมีความเสี่ยงสูงที่จะเกิดเหตุการณ์ Race Condition ระบบจะทำงานแบบ Asynchronous โดยรับคำสั่งผ่าน Amazon SQS จากนั้น AWS Lambda (Async Worker) จะดึงคำสั่งไปประมวลผลทีละรายการ โดยอาศัยกลไก Transaction และ Locking ของ Amazon RDS เพื่อรับประกันว่าการตัดโควตาความจุจะมีความถูกต้องและไม่เกิดการรับคนเกินกำหนด เมื่อดำเนินการเสร็จสิ้น ระบบจะส่ง Event ยืนยันผลลัพธ์ผ่าน Amazon SNS เพื่อแจ้งให้ระบบต้นทางทราบต่อไป

Service Interaction

Upstream Services (บริการต้นทางที่เรียกใช้งาน Shelter Service)

1) Citizen Mobile/Web App (Citizens / ผู้ประสบภัย) ใช้สำหรับประชาชนในการค้นหาศูนย์พักพิงที่อยู่ใกล้ที่สุดและยังมีสถานะเปิดรับ

- **Public Endpoints**
    - **GET /v1/shelters ใช้ค้นหาและกรองศูนย์พักพิง โดยคำนวณระยะทางจากพิกัดผู้ประสบภัย**

2) Shelter Staff Console (เจ้าหน้าที่ศูนย์พักพิง / Admin) ใช้สำหรับเจ้าหน้าที่ในการดูรายละเอียดศูนย์พักพิง อัปเดตโควตาความจุแบบฉุกเฉิน และลงทะเบียนศูนย์พักพิงแห่งใหม่

- **Staff Endpoints**

- **GET /v1/shelters/{shelterId}** ใช้ดึงข้อมูลรายละเอียดทั้งหมดของศูนย์พักพิงแบบเจาะจง
- **PATCH /v1/shelters/{shelterId}** ใช้สำหรับเจ้าหน้าที่หน้างานในการอัปเดตสถานะความจุ (**capacityAvailable)** หรือแก้ไขสภาพเส้นทาง (**accessLevel)**
- **POST /v1/shelters** ใช้ลงทะเบียนข้อมูลศูนย์พักพิงแห่งใหม่เข้าสู่ระบบ

3) Evacuation / Dispatch Service (ระบบสั่งการและประสานงานอพยพ) ใช้สำหรับส่งคำสั่งขอจองพื้นที่ในศูนย์พักพิงให้ผู้ประสบภัยแบบ **Asynchronous** ป้องกันการแย่งกันจอง

- **Command Channel (Message Queue)**
  - **SQS: shelter.reservation.commands.v1** รับ **Event ShelterReservationRequested** เพื่อนำคำขอจองเข้าคิวประมวลผลตัดยอดโควตาในฐานข้อมูล

Downstream Services (บริการปลายทางที่ **Shelter Service** เรียกหรือส่งข้อมูลไป)

A) Downstream แบบ **Synchronous**

- **Pre-Arrival Notification Service (**นายวชิรกรณ์ ชิน) **Shelter Service** เรียกใช้งาน **Pre-Arrival Notification Service** เพื่อส่งข้อมูลเตรียมความพร้อมล่วงหน้าให้แก่หน่วยแพทย์ประจำศูนย์พักพิง (**Medical Tent)** ในกรณีที่ผู้ประสบภัยเดินทางมาด้วยรถพยาบาล หรือมีเงื่อนไขด้านสุขภาพ
  - ตัวอย่างการเรียก: **POST /v1/notifications**
  - ผลลัพธ์ที่คาดหวัง:
    - **201 Created** เมื่อส่งข้อมูลแจ้งเตือนสำเร็จ
    - **400 Bad Request** กรณีข้อมูลผู้ป่วยไม่ครบถ้วน
    - **503 Service Unavailable** เมื่อบริการปลายทางไม่พร้อมใช้งาน
  - บทบาทหลัก:
    - แจ้งเตือนสถานะการเดินทาง (**EN ROUTE)** ไปยังศูนย์พักพิงปลายทาง
    - ส่งต่อข้อมูลอาการ (**Symptom)** และระดับความฉุกเฉิน (**Triage Level)** เพื่อเตรียมเครื่องมือแพทย์

B) Downstream แบบ **Asynchronous (**ส่งข้อมูลผ่าน **Pub/Sub)**

- **Event Bus (SNS) Shelter Service จะ Publish Domain Events ออกไปยัง Event Bus เพื่อแจ้งผลการจองพื้นที่ ผ่าน Topic:**
    - **SNS: evac.reservation.results.v1**
    - **เหตุการณ์ที่ Publish ได้แก่:**
        - **ShelterReservationConfirmed (จองพื้นที่สำเร็จและตัดโควตาแล้ว)**
        - **ShelterReservationRejected (จองไม่สำเร็จ เช่น ศูนย์เต็มระหว่างคิว)**
- **Manage Dispatch Service / RescueRequest Service (นายภัทรภูมิ กิ่งชัย) รับ Event ผลการจองพื้นที่จาก Shelter Service กลับไปเพื่อทำงานต่อ**
    - **บทบาทหลัก:**
        - **รับทราบว่าผู้ประสบภัยมีที่พักรองรับแน่นอนแล้ว**
        - **อัปเดตสถานะเป้าหมายปลายทางให้รถกู้ภัยเดินทางไปส่งผู้ประสบภัยได้ถูกต้อง**

สรุปภาพรวมการไหลของข้อมูล (Data Flow Summary)

1. ประชาชนเรียกใช้งาน Public Endpoints (GET /v1/shelters) เพื่อค้นหาศูนย์พักพิงที่ใกล้และยังเปิดรับ
2. เจ้าหน้าที่เรียกใช้งาน Staff Endpoints เพื่อดู แก้ไข และเพิ่มข้อมูลศูนย์พักพิงในฐานข้อมูล
3. ระบบรถกู้ภัยหรือศูนย์สั่งการ (Upstream) ส่งคำสั่งขอจองพื้นที่เข้ามาทาง Asynchronous Queue (SQS)
4. เมื่อ Shelter Service ตัดโควตาสำเร็จ จะเรียก API แบบ Synchronous ไปยัง **Pre-Arrival Notification Service** เพื่อแจ้งให้หน่วยแพทย์ของศูนย์พักพิงเตรียมพร้อมรับมือ
5. จากนั้น Shelter Service จะ Publish Event (ShelterReservationConfirmed) กลับไปยัง SNS เพื่อแจ้งผลลัพธ์ให้ **RescueRequest Service / Dispatch Service** ทราบและดำเนินการส่งรถกู้ภัยต่อไป

Dependency Mapping – Shelter Service

1) Amazon RDS (PostgreSQL)

- **Type: Database**
- **Interaction Style: Internal Data Access (Read / Write / Lock)**
- **Purpose:** จัดเก็บข้อมูลหลักที่ **Shelter Service** เป็นเจ้าของ ได้แก่
    - **Shelter Master Data (**ข้อมูลพิกัดและรายละเอียดศูนย์)
    - **Shelter Operational State (**ข้อมูลโควตาความจุและสถานะปัจจุบัน)
    - **Reservation Data (**ประวัติการจองพื้นที่)
    - ใช้ทำ **Row-level Locking (FOR UPDATE)** เพื่อป้องกัน **Race Condition** แบบเด็ดขาด
- **Criticality: Critical (**หัวใจหลักของระบบ)
- **Failure Handling:**
    - ใช้ **Database Transaction** ครอบคลุมการทำงาน หาก **Write/Update** ล้มเหลว ระบบจะทำการ **Rollback** ข้อมูลกลับสู่สถานะเดิมทันที
    - หาก **RDS** ไม่ตอบสนอง (**Timeout/Down) -> ฟังก์ชัน Synchronous** จะตอบ **503 Service Unavailable** และฟังก์ชัน **Asynchronous** จะคืนข้อความกลับเข้า **SQS** เพื่อรอประมวลผลใหม่ (**Retry)**

2) Amazon SQS (shelter.reservation.commands.v1)

- **Type: Message Queue**
- **Interaction Style: Asynchronous (Point-to-Point)**
- **Purpose:** ทำหน้าที่เป็น **Buffer (**กล่องรับบัตรคิว) สำหรับรับคำสั่งจองพื้นที่จาก **Upstream Services** ป้องกันไม่ให้ **Database** ล่มเมื่อมี **Request** ปริมาณมหาศาลสาดเข้ามาพร้อมกัน
- **Criticality: Critical (**สำหรับฟังก์ชันการจอง) / **Non-Critical (**สำหรับฟังก์ชันการค้นหาที่พัก)
- **Failure Handling:**
    - หาก **SQS** ขัดข้องหรือไม่สามารถรับข้อความได้ **-> API** การจองจะปฏิเสธคำขอและตอบ **503 Service Unavailable** ให้ **Client** ทำการ **Retry** ใหม่ในภายหลัง

- ข้อความที่ **Lambda Worker** ประมวลผลล้มเหลวเกินจำนวนครั้งที่กำหนด (เช่น **Error** จากโค้ดหรือข้อมูลผิดปกติ) จะถูกส่งไปยัง **Dead Letter Queue (DLQ)** เพื่อไม่ให้บล็อกคิวหลัก

3) Amazon SNS (evac.reservation.results.v1)

- **Type: Topic / Event Bus**
- **Interaction Style: Asynchronous (Event Pub/Sub)**
- **Purpose:** กระจายเหตุการณ์แจ้งผลลัพธ์การจอง (**ShelterReservationConfirmed** หรือ **ShelterReservationRejected)** กลับไปยังระบบต้นทาง และแจ้งบริการปลายทาง (เช่น **Pre-Arrival Notification Service)** ให้ทำงานต่อ
- **Criticality: Non-Critical (**ต่อการตัดยอดโควตาในฐานข้อมูล) / **Critical (**ต่อ **Workflow** ของระบบปลายทางที่รอผลลัพธ์)
- **Failure Handling:**
  - หาก **Publish event** ไม่สำเร็จ **->** ระบบจะบันทึก **Log/Metric** และทำการ **Retry**
  - เนื่องจากการอัปเดตสถานะใน **Database** ถือเป็น **Source of Truth** หากส่ง **Event** ไม่สำเร็จจริง ๆ ระบบปลายทางยังสามารถใช้ **API** แบบ **Synchronous** เข้ามา **Query** ดูผลการจองได้

4) Google Maps API / Location Service (ถ้ามีการนำมาใช้เสริมในอนาคต)

- **Type: External 3rd Party API**
- **Interaction Style: Synchronous (REST)**
- **Purpose:** ใช้แปลงพิกัด (**Lat/Long)** เป็นที่อยู่ หรือดึงข้อมูลสภาพการจราจรแบบ **Real-time** เพื่อนำมาคำนวณ **Access Level**
- **Criticality: Non-Critical**
- **Failure Handling:**
  - หาก **API** ไม่ตอบสนอง หรือ **Timeout ->** ระบบจะข้ามการคำนวณข้อมูลจราจร และใช้สมการ **Haversine** คำนวณระยะทางกระจัด (**Straight-line distance)** จากพิกัดที่มีอยู่ใน **Database** ทดแทน เพื่อให้ระบบยังตอบสนองต่อผู้ใช้ได้ (**Graceful Degradation)**